

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 - PENCARIAN NILAI MAX/MIN



NAMA : AIS NURHANIFAH

NIM : 109082500162

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pencarian nilai maksimum dan minimum merupakan algoritma dasar dalam pemrograman yang berfungsi untuk menemukan nilai tertinggi dan terendah di dalam sekumpulan data. Logika dasarnya bekerja dengan menetapkan elemen pertama dari sebuah *array* sebagai nilai ekstrem awal. Selanjutnya, program akan melakukan perulangan (iterasi) untuk membandingkan nilai awal tersebut dengan elemen-elemen berikutnya secara berurutan di dalam *array*. Jika program menemukan elemen yang nilainya lebih besar (untuk pencarian maksimum) atau lebih kecil (untuk pencarian minimum), maka nilai ekstrem sementara tersebut akan diperbarui.

Dalam implementasinya menggunakan bahasa pemrograman Go, algoritma ini sangat mengandalkan penggunaan struktur kontrol seperti perulangan `for` dan percabangan kondisional `if`. Bahasa Go juga memberikan kemudahan melalui pembuatan subprogram atau fungsi (*function*) yang mampu mengembalikan lebih dari satu nilai (*multiple return values*). Hal ini sangat berguna untuk mengembalikan nilai ekstrem beserta posisi indeksinya secara bersamaan. Selain itu, pencarian nilai ini juga dapat dikombinasikan dengan penggunaan *pointer* agar manipulasi data minimum dan maksimum di dalam subprogram dapat langsung mengubah variabel di memori utama.

Algoritma pencarian nilai maksimum dan minimum tidak hanya terbatas pada tipe data primitif, tetapi juga dapat diterapkan pada tipe data bentukan (struktur data kompleks) seperti `struct`. Sebagai contoh, apabila data disimpan di dalam *array of struct* yang berisi informasi kompleks (seperti nama, NIM, dan IPK mahasiswa), perbandingan nilai ekstrem dapat ditargetkan pada satu atribut spesifik saja, misalnya membandingkan atribut IPK antarmahasiswa. Fleksibilitas ini memungkinkan pengelolaan serta analisis rekaman data yang saling terkait menjadi lebih terstruktur dan efisien.

B. Guided

1. [minmaxArray.go]

```
package main

import "fmt"

func nilaiTerbesar(array []int) (int, int) {
    var idx int
    var nilaiTerbesar int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] > nilaiTerbesar {
            nilaiTerbesar = array[idx]
            idxDitemukan = idx
        }
    }
}
```

```

    }
    return nilaiTerbesar, idxDitemukan
}

func nilaiTerkecil(array []int) (int, int) {
    var idx int
    var nilaiTerkecil int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] < nilaiTerkecil {
            nilaiTerkecil = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerkecil, idxDitemukan
}

func main() {
    var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

    var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
    var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])

    fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d",
nilaiMin, idxNilaiMin)
    fmt.Println()
    fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d",
nilaiMaks, idxNilaiMaks)
}

```

Output :



```

PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Wekk 9 - Modul 10\Guided 1.go"
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini memiliki dua fungsi utama yaitu nilaiTerbesar dan nilaiTerkecil yang digunakan untuk mencari nilai terbesar dan terkecil dalam sebuah array. Kedua fungsi bekerja dengan cara melakukan perulangan untuk membandingkan setiap elemen dalam array, lalu menyimpan nilai yang paling besar atau paling kecil beserta indeksinya. Jika ditemukan nilai yang lebih besar atau lebih kecil dari sebelumnya, maka nilai dan indeks tersebut akan diperbarui. Pada akhirnya,

masing-masing fungsi mengembalikan dua nilai, yaitu nilai hasil (terbesar atau terkecil) dan posisi (indeks) ditemukannya nilai tersebut dalam array.

Pada bagian main, program mendefinisikan sebuah array berisi lima angka, yaitu 45, 23, 98, 66, dan 12. Kemudian program memanggil kedua fungsi tersebut untuk mendapatkan nilai terbesar dan terkecil beserta indeksinya. Hasil yang diperoleh disimpan dalam variabel, lalu ditampilkan ke layar menggunakan `fmt.Printf`. Output yang dihasilkan menunjukkan nilai terkecil beserta posisinya, dan nilai terbesar beserta posisinya dalam array. Program ini membantu pengguna mengetahui tidak hanya nilai ekstrem dalam data, tetapi juga letak posisinya.

2. [minmaxStruct.go]

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
```

```

var idxDitemukan int = 0
var i int

for i = 1; i < len(array); i++ {
    if array[i].IPK < mhsTerkecil.IPK {
        mhsTerkecil = array[i]
        idxDitemukan = i
    }
}

return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("=====")
    }

    fmt.Println()
    for j = 0; j < len(arrMhs); j++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", j)
        fmt.Println("Nama : ", arrMhs[j].nama)
        fmt.Println("NIM : ", arrMhs[j].nim)
        fmt.Println("IPK : ", arrMhs[j].IPK)
        fmt.Println("=====")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerb Besar(arrMhs[:])
    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")

```

```

        fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim,
idxMaks)
        fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim,
idxMin)
    }
}

```

Output :

```

PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Wekk 9 - Modul 10\Guided 2.go"
DATA MAHASISWA INDEKS KE- 0
Masukkan nama : Ais
Masukkan NIM :109082500162
Masukkan IPK : 3.9
=====
DATA MAHASISWA INDEKS KE- 1
Masukkan nama : Cesia
Masukkan NIM :109082500178
Masukkan IPK : 3.8
=====
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : Iyan
Masukkan NIM :109082500193
Masukkan IPK : 3.7
=====

=====

DATA MAHASISWA INDEKS KE- 0
Nama : Ais
NIM : 109082500162
IPK : 3.9
=====
DATA MAHASISWA INDEKS KE- 1
Nama : Cesia
NIM : 109082500178
IPK : 3.8
=====
DATA MAHASISWA INDEKS KE- 2
Nama : Iyan
NIM : 109082500193
IPK : 3.7
=====

=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 3.90, atas nama Ais dengan NIM 109082500162, ditemukan pada indeks ke-0IPK Terkecil : 3.70,
atas nama Iyan dengan NIM 109082500193, ditemukan pada indeks ke-2
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini digunakan untuk menyimpan dan mengolah data beberapa mahasiswa yang terdiri dari **nama, NIM, dan IPK** menggunakan tipe data struct. Data mahasiswa disimpan dalam sebuah array dengan kapasitas 3. Pada bagian awal (main), program meminta pengguna untuk memasukkan data tiap mahasiswa satu per satu, lalu menampilkannya kembali sebagai bentuk pengecekan data. Setelah itu, program memanfaatkan dua fungsi, yaitu IPKTerbesar dan IPKTerkecil, untuk mencari mahasiswa dengan IPK tertinggi dan

terendah. Kedua fungsi ini bekerja dengan cara membandingkan nilai IPK setiap elemen dalam array dan menyimpan data mahasiswa yang memenuhi kriteria tersebut beserta indeksinya.

Hasil dari proses tersebut berupa informasi mahasiswa dengan IPK terbesar dan terkecil, lengkap dengan nama, NIM, serta posisi indeksinya dalam array. Secara keseluruhan, program ini membantu dalam mengelola data mahasiswa sekaligus mengetahui nilai IPK tertinggi dan terendah dari data yang dimasukkan.

C. Unguided

1. Soal Unguided 1

Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual. Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya Nbilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual. Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    var berat [1000]float64

    fmt.Print("Masukkan jumlah kelinci: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Berat kelinci ke-%d: ", i+1)
        fmt.Scan(&berat[i])
    }

    min := berat[0]
    max := berat[0]

    for i := 1; i < n; i++ {
        if berat[i] < min {
```

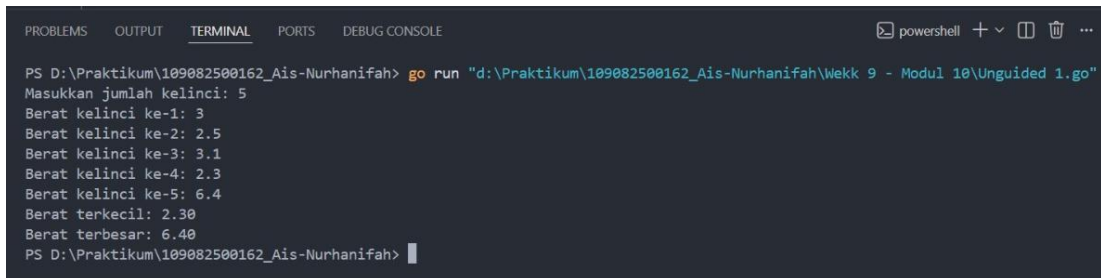
```

        min = berat[i]
    }
    if berat[i] > max {
        max = berat[i]
    }
}

fmt.Printf("Berat terkecil: %.2f\n", min)
fmt.Printf("Berat terbesar: %.2f\n", max)
}

```

Output :



```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Wekk 9 - Modul 10\Unguided 1.go"
Masukkan jumlah kelinci: 5
Berat kelinci ke-1: 3
Berat kelinci ke-2: 2.5
Berat kelinci ke-3: 3.1
Berat kelinci ke-4: 2.3
Berat kelinci ke-5: 6.4
Berat terkecil: 2.30
Berat terbesar: 6.40
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini digunakan untuk mendata berat beberapa kelinci menggunakan array dengan kapasitas hingga 1000 data. Di awal, program meminta jumlah kelinci (n), kemudian pengguna memasukkan berat masing-masing kelinci satu per satu. Setiap nilai berat yang dimasukkan akan disimpan ke dalam array untuk diproses lebih lanjut, sehingga seluruh data berat kelinci dapat dikumpulkan terlebih dahulu sebelum dilakukan perhitungan.

Setelah semua data tersimpan, program mulai menentukan nilai berat terkecil dan terbesar. Nilai awal minimum dan maksimum diambil dari data pertama, kemudian program melakukan perulangan untuk membandingkan setiap data berikutnya. Jika ditemukan berat yang lebih kecil, maka nilai minimum diperbarui, dan jika ditemukan yang lebih besar, maka nilai maksimum juga diperbarui hingga seluruh data selesai diperiksa.

Hasil akhirnya berupa informasi berat kelinci paling kecil dan paling besar dari seluruh data yang dimasukkan. Contoh hasil yang diperoleh misalnya: *Masukkan jumlah kelinci: 5*, kemudian *Berat kelinci ke-1: 3, ke-2: 2.5, ke-3: 3.1, ke-4: 2.3, ke-5: 6.4*, sehingga didapatkan *Berat terkecil: 2.30* dan *Berat terbesar: 6.40*. Dengan demikian, pengguna dapat mengetahui perbandingan berat kelinci paling ringan dan paling berat dari data tersebut.

2. Soal Unguided 2

Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program dijual. ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual. Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

Jawaban :

```
package main

import "fmt"

func main() {
    var x, y int
    var ikan [1000]float64

    fmt.Println("Masukkan jumlah ikan dan kapasitas per wadah: ")
    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Printf("Berat ikan ke-%d: ", i+1)
        fmt.Scan(&ikan[i])
    }

    var totalWadah []float64

    for i := 0; i < x; i += y {
        var jumlah float64 = 0

        for j := i; j < i+y && j < x; j++ {
            jumlah += ikan[j]
        }

        totalWadah = append(totalWadah, jumlah)
    }
}
```

```

    fmt.Println("Total berat tiap wadah:")
    for i := 0; i < len(totalWadah); i++ {
        fmt.Printf("%.2f ", totalWadah[i])
    }
    fmt.Println()

    var total float64 = 0
    for i := 0; i < len(totalWadah); i++ {
        total += totalWadah[i]
    }

    rata := total / float64(len(totalWadah))

    fmt.Printf("Rata-rata berat per wadah: %.2f\n", rata)
}

```

Output :



```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Wekk 9 - Modul 10\Unguided 2.go"
Masukkan jumlah ikan dan kapasitas per wadah: 5 2
Berat ikan ke-1: 1
Berat ikan ke-2: 2
Berat ikan ke-3: 3
Berat ikan ke-4: 4
Berat ikan ke-5: 5
Total berat tiap wadah:
3.00 7.00 5.00
Rata-rata berat per wadah: 5.00
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini digunakan untuk mengelompokkan berat ikan ke dalam beberapa wadah berdasarkan kapasitas yang ditentukan. Di awal, program meminta dua input yaitu jumlah ikan (x) dan kapasitas tiap wadah (y). Setelah itu, pengguna memasukkan berat masing-masing ikan satu per satu, dan semua data tersebut disimpan dalam array untuk diproses. Selanjutnya, program menghitung total berat ikan pada setiap wadah. Proses ini dilakukan dengan mengambil data ikan sebanyak kapasitas wadah (y) secara berurutan, lalu menjumlahkannya.

Hasil penjumlahan tiap kelompok disimpan dalam sebuah slice (totalWadah). Jika jumlah ikan tidak habis dibagi kapasitas wadah, maka wadah terakhir akan berisi sisa ikan yang ada. Hasil akhirnya berupa total berat ikan di setiap wadah serta rata-rata berat per wadah. Contohnya: *Masukkan jumlah ikan dan kapasitas per wadah: 5 2*, lalu berat ikan yang dimasukkan adalah 1, 2, 3, 4, 5, sehingga total tiap wadah menjadi 3.00, 7.00, 5.00 dan rata-ratanya adalah 5.00. Dengan demikian, program ini membantu mengetahui distribusi berat ikan dalam setiap wadah dan nilai rata-ratanya.

3. Soal Unguided 3

Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya. Buatlah program dengan spesifikasi subprogram sebagai berikut:

```
type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita; bMin, bMax *float64) {
  /* I.S. Terdefinisi array dinamis arrBerat
   Proses: Menghitung berat minimum dan maksimum dalam array
   F.S. Menampilkan berat minimum dan maksimum balita */
  ...
}

function rerata (arrBerat arrBalita) real {
  /* menghitung dan mengembalikan rerata berat balita dalam array */
  ...
}
```

Perhatikan sesi interaksi pada contoh berikut ini (teks bergaris bawah adalah input/read)

```
Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38kg
```

Jawaban :

```
package main

import "fmt"

type arrBalita [100]float64

func hitungMinMax(arr arrBalita, n int, bMin, bMax *float64) {
  *bMin = arr[0]
  *bMax = arr[0]

  for i := 1; i < n; i++ {
    if arr[i] < *bMin {
```

```

        *bMin = arr[i]
    }
    if arr[i] > *bMax {
        *bMax = arr[i]
    }
}

func rerata(arr arrBalita, n int) float64 {
    var total float64 = 0
    for i := 0; i < n; i++ {
        total += arr[i]
    }
    return total / float64(n)
}

func main() {
    var data arrBalita
    var n int
    var min, max float64

    fmt.Print("Masukkan banyak data berat balita: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukkan berat balita ke-%d: ", i+1)
        fmt.Scan(&data[i])
    }

    hitungMinMax(data, n, &min, &max)

    rata := rerata(data, n)

    fmt.Printf("Berat balita minimum: %.2f kg\n", min)
    fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
    fmt.Printf("Rerata berat balita: %.2f kg\n", rata)
}

```

Output :

```

PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Wekk 9 - Modul 10\Unguided 3.go"
Masukkan banyak data berat balita: 3
Masukkan berat balita ke-1: 5.3
Masukkan berat balita ke-2: 6.2
Masukkan berat balita ke-3: 4.1
Berat balita minimum: 4.10 kg
Berat balita maksimum: 6.20 kg
Rerata berat balita: 5.20 kg
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini digunakan untuk mengolah data berat balita dengan menggunakan array bertipe `arrBalita`. Di awal, program meminta jumlah data (n), lalu pengguna memasukkan berat balita satu per satu. Semua data tersebut disimpan ke dalam array untuk kemudian diproses lebih lanjut. Program juga menggunakan konsep fungsi (subprogram) agar perhitungan lebih terstruktur.

Selanjutnya, program memiliki fungsi `hitungMinMax` untuk mencari berat minimum dan maksimum. Fungsi ini bekerja dengan membandingkan setiap data dalam array, lalu menyimpan nilai terkecil dan terbesar melalui parameter pointer. Selain itu, terdapat fungsi `rerata` yang digunakan untuk menghitung rata-rata berat balita dengan menjumlahkan seluruh data lalu membaginya dengan jumlah data.

Hasil akhirnya berupa informasi berat balita paling kecil, paling besar, dan rata-ratanya. Contohnya: *Masukkan banyak data berat balita: 3*, lalu berat yang dimasukkan *5.3, 6.2, 4.1*, sehingga diperoleh *berat minimum 4.10 kg, maksimum 6.20 kg, dan rata-rata 5.20 kg*. Program ini memudahkan dalam menganalisis data berat balita secara sederhana dan terstruktur.

D. Kesimpulan

Penerapan algoritma pencarian nilai maksimum dan minimum sangat efektif dalam mengolah serta menganalisis sekumpulan data. Penggunaan *array* dengan kapasitas tertentu terbukti sangat penting sebagai media penyimpanan kumpulan masukan data sebelum dieksekusi oleh program. Dengan menjalankan perbandingan kondisional secara iteratif dari awal hingga akhir *array*, program mampu menyeleksi dan menetapkan nilai yang paling ekstrem secara akurat.

Penggunaan subprogram atau fungsi secara terpisah sangat membantu dalam menciptakan struktur kode yang modular, rapi, dan mudah dibaca. Pemisahan fungsi spesifik, seperti fungsi perhitungan maksimum, fungsi perhitungan minimum, serta fungsi rata-rata, membuktikan penerapan praktik pemrograman yang baik.

Penerapan konsep perulangan, *array*, *struct*, dan algoritma Max/Min ini terbukti sangat aplikatif untuk memecahkan permasalahan pencatatan dan perhitungan di dunia nyata. Berbagai studi kasus yang diselesaikan dalam praktikum—mulai dari memfilter IPK mahasiswa, menyeleksi berat kelinci dan wadah ikan, hingga pendataan kesehatan balita di posyandu menunjukkan luasnya cakupan algoritma.